PROFESSIONAL QA GUIDE

Next.js Static Website

How to prove quality using free, established standards and tools.

Six categories of professional testing — each with the best free tools, what to run, what to capture, and how to present findings to a client.

01	02	03	04	05	06
Performance	**Accessibility**	**SEO**	**Security**	**Bundle & Build**	**Cross-Browser**

How to Use This Guide

Each section covers one quality category. For every category you will find:

- The industry standard it maps to (e.g. WCAG 2.2, Lighthouse, OWASP)
- The best free tools — browser-based, CLI, or both
- Exactly what to run and where
- What to save or screenshot as proof
- The thresholds that define a professional result
- How to present the findings to a client

WHY THIS MATTERS A score or report from a recognised third-party tool carries far more weight with a client than a developer's own word. These tools are the same ones used by Google, Mozilla, and enterprise QA teams.

What You Are Aiming For

Category	Minimum	Professional	Exemplary
Performance	Lighthouse ≥ 70	Lighthouse ≥ 90	All CWV Green
Accessibility	0 critical violations	WCAG 2.1 AA pass	WCAG 2.2 AA pass
SEO	Lighthouse ≥ 80	Lighthouse ≥ 90	+ Structured data
Security	HTTPS	All OWASP headers	A+ on securityheaders.com
Bundle & Build	< 200 KB JS	< 100 KB JS	< 60 KB + image opt.
Cross-Browser	Chrome + Firefox	5 browsers, 2 devices	Playwright CI suite

01 — Performance

Core Web Vitals · Lighthouse · PageSpeed Insights · WebPageTest

Why It Matters

Google uses Core Web Vitals as a ranking signal. A slow site loses users — 53% of mobile visitors abandon a page that takes longer than 3 seconds to load. Performance is the most visible proof of technical quality.

The Standard

Google's Core Web Vitals (CWV) are the official web performance standard, measured by Lighthouse (the engine behind PageSpeed Insights, Chrome DevTools, and CI pipelines).

Key Metrics & Thresholds

Metric	What It Measures	Good	Needs Work	Poor
LCP — Largest Contentful Paint	How fast the main content loads	≤ 2.5 s	≤ 4 s	> 4 s
INP — Interaction to Next Paint	Responsiveness to user input	≤ 200 ms	≤ 500 ms	> 500 ms
CLS — Cumulative Layout Shift	Visual stability (no jumping)	≤ 0.1	≤ 0.25	> 0.25
TTFB — Time to First Byte	Server/CDN response speed	≤ 800 ms	≤ 1.8 s	> 1.8 s
FCP — First Contentful Paint	When first content appears	≤ 1.8 s	≤ 3 s	> 3 s
TBT — Total Blocking Time	Main thread blocked duration	≤ 200 ms	≤ 600 ms	> 600 ms

Free Tools

Google PageSpeed Insights https://pagespeed.web.dev

The canonical tool. Runs Lighthouse for both mobile and desktop. Provides lab data and real-world CrUX field data. This is the report to share with clients — it is published by Google itself.

Free · **Browser-based** · **No install** · **Official**

Lighthouse (Chrome DevTools / CLI) https://developer.chrome.com/docs/lighthouse

The engine behind PageSpeed Insights. Run it locally in Chrome DevTools (F12 > Lighthouse tab) or via the CLI for automation. Produces an HTML report you can save and share.

Free · **CLI** · **HTML report** · **CI-ready**

WebPageTest https://www.webpagetest.org

Enterprise-grade waterfall analysis with real devices and real networks. Shows filmstrip screenshots, connection timing, and third-party impact. The waterfall chart is compelling proof for clients.

Free · **Browser-based** · **Real devices** · **Waterfall chart**

Chrome User Experience Report (CrUX) https://developer.chrome.com/docs/crux

Real-world field data from Chrome users. Accessed via PageSpeed Insights or the CrUX Dashboard. Shows the 75th percentile experience of actual visitors over 28 days.

Free · **Field data** · **Real users** · **28-day window**

What to Run — Step by Step

1. Go to https://pagespeed.web.dev and enter your production URL.
2. Run the analysis for both Mobile and Desktop tabs.
3. Screenshot or download the full report page — capture all four Lighthouse category scores, and the six CWV metrics with their green/amber/red status.
4. Optionally run WebPageTest for a filmstrip view: go to webpagetest.org, enter your URL, choose 'Simple' and run from a location close to your users.
5. For the CLI report: open Terminal and run the command below, then open the generated HTML file.

CLI COMMAND lighthouse https://yoursite.com --output html --output-path ./lighthouse-report.html --chrome-flags="--headless"

What to Capture as Proof

- Full PageSpeed Insights screenshot — both mobile and desktop
- The four Lighthouse category scores (Performance, Accessibility, Best Practices, SEO)
- The Core Web Vitals panel showing Good/Improvement/Poor per metric
- The saved lighthouse-report.html file (shareable standalone report)
- WebPageTest filmstrip showing page load progression (optional but impressive)

How to Present to a Client

Share the PageSpeed Insights URL directly — it is a public link they can click themselves. A score of 90+ in green is immediately understandable without technical explanation. For context, note that Google's own guidance defines 90–100 as 'Good'.

02 — Accessibility

WCAG 2.1 / 2.2 AA · axe-core · WAVE · Lighthouse

Why It Matters

Accessibility is a legal requirement in most jurisdictions (ADA in the US, EN 301 549 in the EU, Equality Act in the UK). Beyond compliance, it widens your audience and is increasingly a client procurement requirement.

The Standard

WCAG (Web Content Accessibility Guidelines) 2.1 Level AA is the internationally recognised baseline. WCAG 2.2 AA is the current version and should be the target for new builds. It is published by the W3C — the organisation that defines web standards.

Core WCAG Principles

Principle	What It Means	Example Requirement
Perceivable	Content must be perceptible to all users	All images have alt text
Operable	UI must be navigable by all users	Keyboard-only navigation works
Understandable	Content and UI must be clear	Error messages are descriptive
Robust	Content works with assistive tech	Valid HTML, correct ARIA roles

Free Tools

axe DevTools (browser extension) https://www.deque.com/axe/devtools/

The industry standard for automated accessibility testing. Powers Lighthouse's accessibility audit. The free browser extension (Chrome/Firefox) finds and explains violations with direct WCAG references. Run it on every page.

Free · **Browser extension** · **WCAG-mapped** · **Industry standard**

WAVE Web Accessibility Evaluator https://wave.webaim.org

Visual overlay tool from WebAIM. Shows errors, alerts, and structural elements directly on the page. Excellent for presenting to clients — the visual annotations make issues immediately obvious.

Free · **Browser-based** · **Visual overlay** · **No install needed**

Lighthouse Accessibility Audit https://developer.chrome.com/docs/lighthouse

The Lighthouse accessibility score (via PageSpeed Insights or DevTools) is powered by axe-core. Score of 100 = all automated checks pass. Automated checks cover ~30–40% of WCAG criteria.

Free · **Automated** · **Score 0–100** · **Part of PageSpeed**

WebAIM Contrast Checker https://webaim.org/resources/contrastchecker/

Instantly checks foreground/background colour combinations against WCAG 1.4.3 (normal text: 4.5:1) and 1.4.11 (UI elements: 3:1). Enter hex codes and get an immediate pass/fail.

Free · **Browser-based** · **Instant result** · **Hex code input**

Accessibility Insights for Web https://accessibilityinsights.io/docs/web/overview

Microsoft's free tool. The FastPass mode runs automated checks and guides a manual tab-stop test. Full Assessment mode walks through a structured WCAG 2.1 AA audit with guided manual tests. Produces an exportable report.

Free · **Browser extension** · **Manual guided tests** · **Exportable report**

What to Run — Step by Step

6. Install the axe DevTools browser extension from the Chrome Web Store or Firefox Add-ons.
7. Open your site, press F12 to open DevTools, go to the 'axe DevTools' tab, and click 'Scan ALL of my page'.
8. Record the violation count by severity: Critical, Serious, Moderate, Minor. Zero Critical and Serious violations is the professional target.
9. Run WAVE on the same page at wave.webaim.org — paste your URL or use the browser extension.
10. Check all text/background colour pairs using webaim.org/resources/contrastchecker.
11. Test keyboard navigation: press Tab through the page and verify every interactive element is reachable and has a visible focus ring.

What to Capture as Proof

- axe DevTools scan results — screenshot of the summary showing 0 critical/serious violations
- WAVE report page — the summary counts panel
- Contrast checker results for your primary text/background pairs
- Lighthouse Accessibility score from PageSpeed Insights
- Accessibility Insights HTML export (if full assessment was done)

How to Present to a Client

Show the Lighthouse Accessibility score (100 is achievable and impressive). Follow with the axe DevTools screenshot showing zero critical violations. Note which WCAG version you tested against (2.1 AA or 2.2 AA) — this signals professional rigour.

KEY POINT Automated tools catch ~30–40% of WCAG issues. A professional audit also includes manual testing. Note this honestly: 'Automated checks pass; additional manual testing conducted for keyboard navigation and screen reader compatibility.'

03 — SEO

Lighthouse · Google Search Console · Schema.org · Screaming Frog

Why It Matters

SEO is not just about rankings — it is about whether the site can be found, indexed, and understood correctly by search engines and social platforms. A technically sound SEO foundation is table-stakes for any professional web delivery.

The Standard

There is no single ISO standard for SEO, but Google's documentation, Lighthouse's SEO audit rules, and Schema.org structured data specifications are the de facto industry standards. These are what agencies and enterprises test against.

Free Tools

Google Search Console https://search.google.com/search-console

The official tool from Google for monitoring how your site appears in search. Shows indexing status, crawl errors, Core Web Vitals field data, and search performance. Essential for any professional delivery — verify the site is indexed correctly.

Free · **Official Google tool** · **Indexing status** · **Requires site ownership**

Lighthouse SEO Audit https://pagespeed.web.dev

The Lighthouse SEO score checks technical fundamentals: title tags, meta descriptions, robots.txt, canonical tags, mobile-friendliness, and link text quality. Score of 100 = all automated checks pass. Available inside PageSpeed Insights.

Free · **Automated** · **Score 0–100** · **Part of PageSpeed**

Google Rich Results Test https://search.google.com/test/rich-results

Tests whether your pages are eligible for rich results (star ratings, FAQs, breadcrumbs, etc.) based on their JSON-LD structured data. Official Google validator — a pass here is direct proof that your structured data is correct.

Free · **Official Google tool** · **JSON-LD validation** · **No install**

Screaming Frog SEO Spider (free tier) https://www.screamingfrog.co.uk/seo-spider/

Crawls up to 500 URLs for free. Finds broken links (404s), missing meta tags, duplicate titles, redirect chains, and missing alt text across the entire site — not just one page. Used by professional SEO agencies worldwide.

Free up to 500 URLs · **Desktop app** · **Full site crawl** · **Exportable CSV**

Schema Markup Validator https://validator.schema.org

Official Schema.org validator. Checks that your JSON-LD structured data (Organization, WebSite, BreadcrumbList, etc.) is correctly structured according to the vocabulary. A clean result here signals professional implementation.

Free · **Official Schema.org tool** · **JSON-LD** · **No install**

OpenGraph.xyz / Social Share Preview https://www.opengraph.xyz

Shows exactly how your pages will appear when shared on LinkedIn, Facebook, Twitter/X, and Slack. Checks og:title, og:description, og:image, and Twitter card tags. Extremely useful proof for clients who care about social presence.

Free · **Browser-based** · **Social previews** · **No install**

What to Run — Step by Step

12. Run PageSpeed Insights on your key pages and screenshot the SEO score (top right of the Lighthouse results).
13. Go to https://search.google.com/test/rich-results and test your homepage and any other key pages. Screenshot the result — especially if it detects valid structured data.
14. Check your og: tags and social previews at opengraph.xyz — paste your URL and screenshot the social card previews.
15. Manually verify: view-source the page and confirm title, meta description, canonical, and lang attribute are present and correct.
16. Run Screaming Frog on the full site to confirm: zero 404 errors, all pages have titles and meta descriptions, no redirect chains, no duplicate titles.
17. Submit to Google Search Console (requires adding a DNS TXT record or meta tag to verify ownership). Check the Coverage report for indexing errors.

Core SEO Checklist

Check	Tool	Target
Lighthouse SEO score	PageSpeed Insights	100
Title tag — unique, 50–60 chars	Screaming Frog / view-source	Present on every page
Meta description — 120–160 chars	Screaming Frog	Present on every page
Canonical tag	view-source	Present, self-referencing

robots.txt exists and is valid	yoursite.com/robots.txt	Accessible, not blocking CSS/JS
sitemap.xml linked in robots.txt	yoursite.com/sitemap.xml	All pages listed
JSON-LD structured data	Rich Results Test	Valid, no errors
Open Graph tags	OpenGraph.xyz	og:title, og:image, og:description
No 404 broken links	Screaming Frog	Zero 4xx errors
lang attribute on <html>	view-source	Correct language code

What to Capture as Proof

- PageSpeed Insights SEO score screenshot
- Google Rich Results Test screenshot (showing valid structured data)
- OpenGraph.xyz social card previews
- Screaming Frog crawl summary — no 4xx errors, all pages have titles/descriptions
- Google Search Console Coverage report showing pages indexed successfully

🛡 04 — Security

SecurityHeaders.com · Mozilla Observatory · npm audit · OWASP

Why It Matters

Security is the category clients most associate with professionalism and liability. HTTP security headers are low-effort and high-impact — they prevent a class of common attacks and are part of the OWASP standard that enterprise procurement teams check against.

The Standard

OWASP (Open Web Application Security Project) publishes the industry's most recognised web security guidance. Scott Helme's Security Headers grading system is the de facto external check for HTTP header configuration — an A+ rating is achievable on any static site with proper configuration.

Free Tools

SecurityHeaders.com https://securityheaders.com

The most widely referenced HTTP security header checker. Grades your site A+ to F based on the presence and correctness of security headers. An A+ is achievable for any static site and is compelling proof for security-conscious clients. Created by Scott Helme (OWASP contributor).

Free · **Browser-based** · **A+ to F grading** · **Shareable report URL**

Mozilla Observatory https://observatory.mozilla.org

Mozilla's security scanner. Checks HTTP headers, cookies, HTTPS configuration, and redirects. Produces a score out of 130 and a letter grade. Trusted because it comes from Mozilla — the organisation behind Firefox.

Free · **Browser-based** · **Score + letter grade** · **Mozilla-backed**

SSL Labs (Qualys) https://www.ssllabs.com/ssltest/

The gold standard for TLS/HTTPS configuration analysis. Grades your SSL certificate and configuration A+ to F. Checks cipher suites, protocol versions, HSTS, and certificate validity. An A or A+ here proves your HTTPS is properly configured.

Free · **Browser-based** · **A+ to F grading** · **Industry standard**

CSP Evaluator (Google) https://csp-evaluator.withgoogle.com

Google's official Content Security Policy analyser. Paste your CSP header value and it highlights weaknesses and bypasses. A strict, bypass-free CSP is a significant security credential — few sites achieve it.

Free · **Browser-based** · **Official Google tool** · **CSP-specific**

npm audit https://docs.npmjs.com/cli/v10/commands/npm-audit

Built into npm. Checks all dependencies against the npm advisory database for known CVEs (Common Vulnerabilities and Exposures). Zero critical and high vulnerabilities is the professional baseline. Run before every deployment.

Free · **CLI** · **Built into npm** · **CVE database**

The OWASP Security Headers

Header	Protects Against	Recommended Value
Strict-Transport-Security	Downgrade attacks, HTTP interception	max-age=31536000; includeSubDomains; preload
Content-Security-Policy	XSS, data injection, clickjacking	Define allowlists for scripts, styles, media
X-Frame-Options	Clickjacking	DENY or SAMEORIGIN
X-Content-Type-Options	MIME sniffing attacks	nosniff
Referrer-Policy	Referrer information leakage	strict-origin-when-cross-origin
Permissions-Policy	Browser feature abuse	Disable unused APIs (camera, mic, geolocation)
Cross-Origin-Opener-Policy	Cross-origin attacks, Spectre	same-origin

Setting Headers in Next.js

For a static export hosted on Vercel, Netlify, or Cloudflare Pages, security headers are set in the platform configuration — not at the Next.js application level.

Vercel (vercel.json)

```
{
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Frame-Options", "value": "DENY" },
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "Permissions-Policy", "value": "camera=(), microphone=(),
geolocation=()" }
```

```
      ]
    }
  ]
}
```

Netlify (_headers file)

```
/*
  X-Frame-Options: DENY
  X-Content-Type-Options: nosniff
  Referrer-Policy: strict-origin-when-cross-origin
  Permissions-Policy: camera=(), microphone=(), geolocation=()
```

What to Run — Step by Step

18. Go to securityheaders.com and enter your production URL. Screenshot the grade and the header breakdown.
19. Go to observatory.mozilla.org, enter your URL, and screenshot the score and recommendations.
20. Go to ssllabs.com/ssltest/, enter your domain, and screenshot the SSL grade.
21. In your project terminal, run: npm audit — screenshot or copy the vulnerability summary table.
22. If you have a CSP header, test it at csp-evaluator.withgoogle.com.

What to Capture as Proof

- SecurityHeaders.com grade (ideally A+) — the shareable report URL is strong proof
- Mozilla Observatory score screenshot
- SSL Labs grade screenshot
- npm audit output: 'found 0 vulnerabilities' or summary of patched issues

05 — Bundle & Build

Next.js Bundle Analyzer · bundlephobia · Lighthouse · Chrome DevTools

Why It Matters

Bundle size directly affects performance, especially on mobile networks. A bloated JavaScript bundle is the most common cause of poor Lighthouse Performance scores. Proving that your build is lean demonstrates genuine front-end engineering skill.

The Standard

There is no formal ISO standard for bundle size. The benchmarks come from Google's Lighthouse performance budget guidance, Vercel's Next.js build output, and industry consensus from the web performance community (web.dev, CSS-Tricks, Smashing Magazine).

Target Thresholds

Metric	Good	Acceptable	Investigate
First Load JS (total per page)	< 100 KB gzipped	< 200 KB	> 200 KB
Largest JS chunk	< 50 KB gzipped	< 100 KB	> 100 KB
Shared / framework chunk	< 75 KB gzipped	< 120 KB	> 120 KB
CSS total (gzipped)	< 20 KB	< 50 KB	> 50 KB
Total image payload (per page)	< 300 KB	< 800 KB	> 1 MB
Web fonts total	< 40 KB	< 100 KB	> 100 KB

Free Tools

@next/bundle-analyzer https://www.npmjs.com/package/@next/bundle-analyzer

The official Next.js bundle analyser. Adds an interactive treemap to your build process — click into any chunk to see exactly which npm packages contribute to its size. The most direct way to find and fix bundle bloat.

Free · **npm package** · **Interactive treemap** · **Official Next.js tool**

Bundlephobia https://bundlephobia.com

Shows the size (minified + gzipped) of any npm package before you add it to your project. Also suggests lighter alternatives. Useful for proving that your dependency choices were deliberate and size-conscious.

Free · **Browser-based** · **No install** · **Per-package analysis**

Chrome DevTools — Coverage https://developer.chrome.com/docs/devtools/coverage

Built into Chrome. Open DevTools (F12), press Ctrl+Shift+P, type 'Coverage', and run. Shows exactly which bytes of each JS and CSS file are actually used on the current page. Red = unused bytes. Unused JS above 50% signals a tree-shaking problem.

Free · **Built into Chrome** · **No install** · **Per-file breakdown**

Chrome DevTools — Network tab https://developer.chrome.com/docs/devtools/network

The definitive tool for measuring actual payload. Filter by JS, CSS, Img, Font, and document type. Enable 'Use large request rows' and look at the Transferred column (compressed) vs Size column (uncompressed). Screenshot this for the client report.

Free · **Built into Chrome** · **No install** · **Real network data**

Next.js Build Output (next build) https://nextjs.org/docs/app/building-your-application/deploying/static-exports

Running next build prints a per-page size table in the terminal showing First Load JS, the page chunk size, and the shared chunk. This is built-in proof from the framework itself. Screenshot the terminal output.

Free · **Built into Next.js** · **No install** · **Per-page breakdown**

What to Run — Step by Step

23. Run next build in your project terminal. Screenshot the entire size table output — it shows every page with its JS size.
24. Install and run the bundle analyser:

```
npm install --save-dev @next/bundle-analyzer
```

Add to next.config.js:

```
const withBundleAnalyzer = require('@next/bundle-analyzer')({ enabled:
process.env.ANALYZE === 'true' })
module.exports = withBundleAnalyzer(nextConfig)
```

Then run:

```
ANALYZE=true npm run build
```

25. Open Chrome DevTools > Network tab. Hard reload the page (Ctrl+Shift+R). Screenshot the totals row at the bottom — it shows total transferred size.
26. Open DevTools > Coverage. Reload the page. Screenshot the coverage report showing % of JS used vs unused.

27. For any large dependency you added, check its size first at bundlephobia.com.

What to Capture as Proof

- next build terminal output — the per-page size table
- Bundle analyser treemap screenshot showing chunk breakdown
- Chrome DevTools Network tab showing total transferred payload
- Coverage report showing high JS utilisation (low unused bytes)

06 — Cross-Browser & Responsive

BrowserStack Free · Playwright · Chrome DevTools · Responsively

Why It Matters

A website that only works in one browser is not a professional deliverable. Responsive design at multiple viewport sizes is a fundamental expectation. Cross-browser testing proves that the site works for all of the client's users, not just developers on MacBooks.

The Standard

The browser support matrix is based on global market share data from StatCounter and caniuse.com. Responsive design standards follow WCAG 1.4.4 (Resize Text) and 1.4.10 (Reflow), which require the site to work at 200% zoom and on a 320px wide viewport without horizontal scrolling.

Browser Support Matrix — Professional Minimum

Browser	Market Share (approx.)	Desktop	Mobile	Priority
Chrome	~65%	Test	Test	Critical
Safari	~19%	Test	Test (iOS)	Critical
Firefox	~5%	Test	Test	High
Edge	~4%	Test	—	High
Samsung Internet	~2.5%	—	Test	Medium

Free Tools

Chrome DevTools — Device Mode https://developer.chrome.com/docs/devtools/device-mode

Built into Chrome. Press Ctrl+Shift+M (or the phone icon in DevTools). Simulates any screen size with preset device profiles (iPhone 14, Pixel 7, iPad, etc.). Throttles network and CPU. The fastest way to check responsive layouts at any breakpoint.

Free · **Built into Chrome** · **No install** · **Device presets**

Firefox Responsive Design Mode https://firefox-source-docs.mozilla.org/devtools-user/responsive_design_mode/

Firefox's equivalent of Chrome's Device Mode. Press Ctrl+Shift+M. Includes its own device presets and can simulate touch events. Useful for catching Firefox-specific rendering differences alongside responsive layout testing.

Free · **Built into Firefox** · **No install** · **Touch simulation**

Responsively App https://responsively.app

Free, open-source browser that shows your site at multiple viewports simultaneously. Side-by-side view of mobile, tablet, and desktop in one window — scrolling is synchronised across all viewports. Produces comparison screenshots perfect for client reports.

Free · **Open source** · **Desktop app** · **Side-by-side view**

BrowserStack (free Spark plan) https://www.browserstack.com

The industry standard for real-device cross-browser testing. The free Spark plan gives limited access to real browsers and devices. A BrowserStack test on a real iPhone Safari and real Android Chrome is the strongest possible cross-browser proof for a client.

Free tier available · **Real devices** · **Industry standard** · **Screenshot capture**

Can I Use https://caniuse.com

The definitive browser compatibility reference. Check whether any CSS property, HTML element, or JavaScript API you used is supported across your target browsers. Use it to justify your technology choices and document any polyfills.

Free · **Browser-based** · **Reference database** · **No install**

Playwright (free, open source) https://playwright.dev

Microsoft's end-to-end testing framework. Automates real Chromium, Firefox, and WebKit (Safari engine) browsers. Can capture screenshots and run visual regression tests at specific viewport sizes. The professional standard for automated cross-browser CI testing.

Free · **Open source** · **Chromium + Firefox + WebKit** · **CI-ready**

Responsive Breakpoints to Test

Breakpoint	Width	Represents	Test Priority
Mobile S	320 px	Smallest common phone (iPhone SE)	Critical
Mobile L	375 px	iPhone 14 / most Android	Critical
Tablet	768 px	iPad portrait	High
Laptop	1280 px	Standard laptop	High
Desktop	1440 px	Common desktop / large laptop	High

Wide	1920 px	Full HD monitor	Medium

What to Run — Step by Step

28. Open Chrome DevTools (F12) > Device Mode (Ctrl+Shift+M). Test each breakpoint in the table above. Check for horizontal overflow, overlapping text, and broken layouts. Screenshot each viewport.
29. Open the same pages in Firefox and Safari. Check for any visual differences (font rendering, flexbox/grid behaviour, form styling).
30. Download Responsively App (responsively.app) and open your site. Screenshot the multi-viewport view.
31. Test keyboard navigation in each browser: Tab through the page, confirm focus indicators are visible, all interactive elements are reachable.
32. For automated testing, install Playwright:

```
npm init playwright@latest
```

Then write a basic viewport test and run:

```
npx playwright test --reporter=html
```

Manual Responsive Checklist

Check	How to Test	Pass Criterion
No horizontal scrollbar at 320px	DevTools Device Mode	document.body.scrollWidth === window.innerWidth
Text readable at 200% zoom	Ctrl+Plus in any browser	No clipped text, no overlap
Viewport meta tag present	view-source	<meta name="viewport" content="width=device-width">
Touch targets ≥ 24×24 px	DevTools inspect element	Computed height/width ≥ 24px
Images scale correctly	Resize browser window	No fixed-width overflow
Navigation usable on mobile	DevTools Device Mode	All nav items accessible
Forms work on mobile keyboard	Real device or simulator	No zoom-on-focus (font ≥ 16px)
Focus indicator visible in all browsers	Tab key in each browser	Visible outline on all focusable elements

What to Capture as Proof

- Responsively App screenshot — multi-viewport side-by-side view

- Chrome DevTools Device Mode screenshots at 320px, 768px, 1280px
- Firefox and Safari screenshots of the same pages
- BrowserStack screenshots from real iPhone Safari and Android Chrome
- Playwright HTML test report (if automated tests written)

Putting It All Together

Once you have run the tests in each category, you have a set of third-party, tool-verified evidence. Here is how to structure that evidence into a client-facing delivery package.

Recommended Delivery Package

Artefact	What to Include	Source
Performance Report	PageSpeed Insights screenshots (mobile + desktop), CWV metrics	PageSpeed Insights URL (shareable)
Accessibility Report	axe scan summary, Lighthouse A11y score, contrast checker results	Screenshots + axe export
SEO Report	Lighthouse SEO score, Rich Results Test, Screaming Frog CSV	Screenshots + CSV export
Security Report	SecurityHeaders.com grade, Mozilla Observatory score, SSL Labs grade, npm audit output	Shareable report URLs
Bundle Report	next build output, Network tab payload, coverage screenshot	Terminal screenshot + DevTools
Cross-Browser Report	Responsively App screenshot, per-browser screenshots, Playwright report	Screenshots + HTML report

The Evidence Principle

CORE PRINCIPLE Every claim in your delivery should be backed by a screenshot or a shareable URL from a recognised third-party tool. 'I tested it' is not proof. 'PageSpeed Insights scores 97 on desktop — here is the public link' is proof.

Quick-Reference Summary Table

Category	Primary Tool	Proof Type	Target Grade / Score
Performance	Google PageSpeed Insights	Shareable URL	≥ 90 (desktop), ≥ 70 (mobile)
Accessibility	axe DevTools + Lighthouse	Screenshot + export	0 critical violations, ≥ 90 Lighthouse
SEO	Lighthouse + Rich Results Test	Screenshot	100 Lighthouse SEO, valid structured data
Security	SecurityHeaders.com	Shareable URL + grade	A or A+

Bundle & Build	next build + Chrome DevTools	Terminal + screenshot	< 100 KB First Load JS
Cross-Browser	Responsively + BrowserStack	Screenshots	Pass in 5 browsers

Further Reading

All tools and standards referenced in this guide are publicly documented. Key resources:

web.dev/learn — Google's free learning platform for web quality

developer.chrome.com/docs — Chrome DevTools documentation

www.w3.org/WAI/WCAG22 — Official WCAG 2.2 specification

owasp.org — OWASP security guidance

web.dev/vitals — Core Web Vitals documentation

nextjs.org/docs — Next.js documentation

End of Guide